



Universidade Estadual de Campinas
Faculdade de Engenharia Mecânica (FEM)
Engenharia de Controle e Automação

ES952 – Trabalho de Graduação II

Relatório Final

Implementação de estratégias de navegação autônoma baseadas
em fusão sensorial para veículos de 4 rodas

Professor Orientador

Prof. Dr. Ely Carneiro de Paiva

Aluno

Bruno Monteiro Bonetti

RA

232488

Julho/2026



Universidade Estadual de Campinas
Faculdade de Engenharia Mecânica (FEM)
Engenharia de Controle e Automação

ES952 – Trabalho de Graduação II

Relatório Final

Implementação de estratégias de navegação autônoma baseadas em fusão sensorial para veículos de 4 rodas

Curso: Engenharia de Controle e Automação

Trabalho de Conclusão de Curso, apresentado à Comissão de Graduação da Faculdade de Engenharia Mecânica, como requisito para a obtenção do título de Engenheiro de Controle e Automação.

Professor Orientador

Prof. Dr. Ely Carneiro de Paiva

Aluno

Bruno Monteiro Bonetti

RA

232488

Campinas, Julho de 2026
S.P. – Brasil

Índice

Resumo	3
Lista de Figuras	4
Lista de Tabelas	5
Lista de Equações	6
Lista de Símbolos	7
1 Introdução	8
1.1 Objetivos	8
1.2 Justificativa	9
2 Revisão Bibliográfica	10
2.1 Veículo Autônomo LIMO AgileX	10
2.1.1 Modos de Operação	10
2.1.2 Computador de Bordo e Software	11
2.1.3 LiDAR — EAI T-Mini Pro	11
2.1.4 Câmera — Orbbec Astra (Dabai U3)	12
2.2 Robot Operating System (ROS)	12
2.2.1 Arquitetura <i>Publish-Subscribe</i>	12
2.2.2 Conceitos Fundamentais	13
2.2.3 Ferramentas Utilizadas	13
2.3 Detecção por Cor (HSV) e Fusão Sensorial	14
2.3.1 Espaço de Cor HSV	14
2.3.2 Fusão Sensorial	14
2.4 YOLO	14
2.4.1 Divisão em Grade e Predições	15
2.4.2 Arquitetura	15
2.4.3 Supressão Não-Máxima (NMS)	16
2.4.4 Função de Custo	16
2.4.5 YOLO neste Projeto	16
2.5 Campos Potenciais	16
2.5.1 Força Atrativa	17
2.5.2 Força Repulsiva	17
2.5.3 Aplicação neste Projeto	17
3 Sistema Desenvolvido	18
3.1 Modo 1 — Seguidor por Cor com Fusão Sensorial	18
3.1.1 Nó 1 — <code>color_detector</code>	18
3.1.2 Nó 2 — <code>lidar_reader</code>	20
3.1.3 Nó 3 — <code>sensor_fusion</code>	21
3.1.4 Nó 4 — <code>follower_controller</code>	21
3.2 Modo 2 — Tarefas com Detecção YOLO	22
3.2.1 Nó — <code>yolo_detector</code>	23
3.2.2 Nó — <code>task_executor</code>	24
3.2.3 Biblioteca <code>robot_primitives</code>	24
3.2.4 Freio de Segurança	24
3.2.5 Campos Potenciais na Aproximação	25
3.2.6 Odometria	25

4	Resultados Experimentais	26
4.1	Seguimento por Cor (Fusão Sensorial)	26
4.1.1	Detecção Visual do Alvo	26
4.1.2	Fusão de Sensores	26
4.1.3	Seguimento Autônomo	26
4.2	Detecção por YOLO	27
4.3	Aproximação com Campos Potenciais	27
5	Conclusão	28
	Referências	29

Resumo

BONETTI, Bruno M. *Implementação de estratégias de navegação autônoma baseadas em fusão sensorial para veículos de 4 rodas*. Faculdade de Engenharia Mecânica, Universidade Estadual de Campinas, Trabalho de Conclusão de Curso, 2026.

Este trabalho aborda o desenvolvimento e a implementação de um sistema de seguimento autônomo de alvos na plataforma robótica AgileX LIMO, utilizando o arcabouço de software ROS. São propostos dois modos de operação complementares. No primeiro, a percepção do alvo se dá por segmentação de cor no espaço HSV, e a fusão sensorial entre a câmera e o sensor LiDAR combina a direção fornecida pela imagem com a distância medida pelo laser, alimentando um controlador proporcional que mantém o robô a uma distância configurável do alvo. No segundo modo, a detecção por cor é substituída pela rede neural de detecção de objetos YOLO, que identifica alvos por categoria, e introduz-se uma camada de execução de tarefas baseada em ações primitivas. Esse modo incorpora ainda um freio de segurança baseado no LiDAR e uma estratégia de aproximação por campos potenciais artificiais, combinando uma força atrativa em direção ao alvo com uma força repulsiva que evita colisões com obstáculos. O sistema foi implementado em Python e validado em *hardware* real, demonstrando a detecção dos alvos, a fusão consistente dos sensores e o comportamento de seguimento e aproximação autônomos.

Palavras-chave: robótica móvel, navegação autônoma, fusão sensorial, LiDAR, YOLO, campos potenciais, ROS.

Lista de Figuras

1	LIMO Pro da AgileX Robotics.	10
2	Varreduras do LiDAR EAI T-Mini Pro em diferentes ambientes.	11
3	Visualização do tópico <code>/camera/rgb/image_raw</code> no Foxglove Studio.	12
4	Padrão <i>publish-subscribe</i> do ROS: o <code>roscore</code> coordena o registro dos nós (linhas tracejadas) e as mensagens trafegam pelo tópico do publicador ao assinante (linhas cheias).	13
5	Pipeline do sistema de seguimento: sensores, nós de processamento e tópicos ROS.	18
6	Processo de calibração HSV para o alvo (copo verde neon).	20
7	Arquitetura do modo de tarefas: percepção por YOLO, leitura do LiDAR e odometria alimentam o <code>task_executor</code> , que aciona as primitivas de movimento em <code>robot_primitives</code>	23
8	Sistema de detecção em operação: <i>bounding box</i> verde em volta do copo com ângulo calculado exibido em tempo real.	26
9	Detecção de um <i>laptop</i> pelo YOLOv3-tiny: <i>bounding box</i> , centro do objeto, classe, confiança e ângulo calculado exibidos em tempo real.	27

Lista de Tabelas

1	Parâmetros de segmentação cromática e geometria da câmera.	19
2	Parâmetros de filtragem e leitura do sensor LiDAR.	21
3	Parâmetros de configuração do nó de fusão sensorial.	21
4	Parâmetros de configuração e controle do sistema de seguimento.	22
5	Parâmetros de configuração do detector YOLO.	23
6	Ações primitivas disponíveis no modo de tarefas.	24
7	Parâmetros da aproximação por campos potenciais.	25

Lista de Equações

Equação 1 — Cinemática do modo diferencial do LIMO.	10
Equação 2 — Máscara binária de segmentação no espaço HSV.	14
Equação 3 — Pontuação de confiança de uma detecção do YOLO.	15
Equação 4 — <i>Intersection over Union</i> (IoU).	15
Equação 6 — Dimensão do tensor de saída do YOLO.	15
Equação 7 — Função de custo do YOLO.	16
Equação 8 — Potencial artificial total (atrativo + repulsivo).	16
Equação 11 — Força atrativa em direção ao alvo.	17
Equação 12 — Potencial repulsivo com raio de influência.	17
Equação 15 — Ângulo do alvo em relação ao centro da imagem.	18
Equação 16 — Filtro de suavização exponencial da fusão sensorial.	21
Equação 17 — Lei de controle proporcional do seguidor.	22
Equação 18 — Condição de acionamento do freio de segurança.	25
Equação 19 — Controle da aproximação por campos potenciais.	25
Equação 20 — Ângulo de guinada (<i>yaw</i>) a partir do quatérnion.	25

Lista de Símbolos

v	Velocidade linear do robô
ω	Velocidade angular do robô
ω_L, ω_R	Velocidades angulares das rodas esquerda e direita
r_w	Raio da roda
b	Distância entre as rodas (bitola)
H, S, V	Canais de matiz, saturação e valor (espaço HSV)
$M(x, y)$	Máscara binária de segmentação por cor
θ_{alvo}	Ângulo do alvo em relação ao centro da imagem
c_x	Posição horizontal do centro do alvo (pixels)
W	Largura da imagem (pixels)
HFOV	Campo de visão horizontal da câmera
α	Fator de suavização do filtro exponencial
e_θ, e_d	Erros angular e de distância
$K_{p,\omega}, K_{p,v}$	Ganhos proporcionais angular e linear (seguidor)
$\Pr(\cdot)$	Probabilidade
IoU	<i>Intersection over Union</i>
S, B, C	Células da grade, caixas por célula e n ^o de classes (YOLO)
$\mathcal{L}$	Função de custo do YOLO
U	Potencial artificial
$\vec{F}$	Força resultante
k_{att}, k_{rep}	Ganhos atrativo e repulsivo (teoria)
ρ, ρ_0	Distância ao obstáculo e raio de influência
k_p, k_R	Ganhos atrativo e repulsivo (implementação)
d	Distância medida pelo LiDAR
d_{alvo}	Distância desejada ao alvo
d_0	Raio de influência da força repulsiva
d_{seg}	Distância de acionamento do freio de segurança
ψ	Ângulo de guinada (<i>yaw</i>)
q_w, q_x, q_y, q_z	Componentes do quatérnion de orientação

1 Introdução

A robótica móvel tem experimentado um crescimento sem precedentes nas últimas décadas, deixando de ser um campo puramente experimental para se tornar uma solução prática em setores como logística, saúde e serviços domésticos. Segundo [1], o sucesso de um sistema autônomo está intrinsecamente ligado à sua capacidade de fechar o ciclo entre percepção, planejamento e controle. Para que um robô opere de forma segura em ambientes compartilhados com seres humanos, ele precisa processar fluxos massivos de dados sensoriais e transformá-los em decisões motoras em milissegundos.

Dentro deste vasto campo, os robôs seguidores (*follow-me robots*) representam um desafio de engenharia particularmente interessante. Diferente de robôs de navegação global que seguem rotas pré-definidas por GPS ou mapas estáticos, o robô seguidor deve lidar com um alvo dinâmico e, muitas vezes, imprevisível. De acordo com [4], a modularidade proporcionada pelo *Robot Operating System* (ROS) é o que permite que desenvolvedores integrem algoritmos complexos de visão e laser de forma eficiente, facilitando a criação de comportamentos reativos como o rastreamento de alvos.

O desafio central da percepção robusta reside nas limitações intrínsecas de cada sensor quando utilizados de forma isolada. Câmeras RGB, embora forneçam dados semânticos valiosos sobre cor e textura, carecem de precisão na medição de profundidade. Conforme discutido por [2], a fusão sensorial surge como a solução para este dilema. Ao combinar a capacidade de identificação da câmera com a precisão métrica do LiDAR, é possível mitigar falhas causadas por variações de iluminação ou oclusões parciais, problemas comuns em cenários reais de operação.

Neste trabalho, utiliza-se a plataforma robótica Agilex LIMO [6], um robô compacto e versátil equipado com múltiplos sensores, para implementar e validar um sistema de seguimento baseado em fusão de dados. O projeto evolui de uma primeira abordagem baseada em detecção de alvo por cor no espaço HSV para uma segunda abordagem baseada em detecção de objetos por categoria com a rede neural YOLO, incorporando ainda campos potenciais para aproximação segura e desvio de obstáculos.

1.1 Objetivos

O objetivo principal desta pesquisa é desenvolver, implementar e testar um pipeline de software capaz de transformar o robô LIMO em um sistema autônomo de seguimento. Para alcançar este objetivo geral, definem-se os seguintes objetivos específicos:

- Desenvolver um nó de processamento visual para detecção de alvos coloridos robusto a ruídos;
- Implementar a leitura sincronizada do sensor LiDAR para extração de distâncias em ângulos específicos;
- Criar um algoritmo de fusão que unifique as coordenadas da câmera e do laser;
- Projetar um controlador proporcional para velocidades lineares e angulares;
- Implementar um comportamento de busca ativa (rotação 360°) para recuperação do alvo em caso de perda de detecção;
- Estender o sistema com detecção de objetos por categoria usando a rede YOLO e uma estratégia de aproximação baseada em campos potenciais com freio de segurança.

1.2 Justificativa

A escolha pela integração entre câmera e LiDAR justifica-se pela busca por redundância e confiabilidade. Enquanto a câmera atua como o "olho" que identifica a natureza do objeto, o LiDAR funciona como o "tato" que define a geometria do ambiente ao redor. Conforme sugerido por [3], sistemas que utilizam LiDAR apresentam uma taxa de sucesso significativamente maior em evitar colisões em ambientes dinâmicos. Portanto, a fusão proposta não apenas melhora a qualidade do seguimento, mas eleva o padrão de segurança operacional do robô em proximidade com obstáculos ou pessoas.

2 Revisão Bibliográfica

2.1 Veículo Autônomo LIMO AgileX

O LIMO é uma plataforma robótica móvel compacta desenvolvida pela AgileX Robotics, projetada para pesquisa e ensino em robótica móvel [8]. Apesar do porte reduzido — ideal para espaços de trabalho limitados —, o robô é equipado com sensores e capacidade computacional suficientes para estudos de navegação autônoma, SLAM, mapeamento, planejamento de trajetórias e desvio de obstáculos. A Figura 1 mostra o modelo LIMO Pro utilizado neste trabalho.



Figura 1: LIMO Pro da AgileX Robotics.

2.1.1 Modos de Operação

Uma característica distintiva do LIMO é sua capacidade de assumir quatro configurações cinemáticas diferentes, selecionáveis mecanicamente por travas na parte frontal do chassi [8]:

- **Diferencial:** quatro rodas em que as duas de um mesmo lado são acionadas em conjunto; a diferença de velocidade entre os lados produz rotação.
- **Esteira:** sobrepõem-se esteiras às rodas, oferecendo maior tração e estabilidade em terrenos irregulares; é controlado como o modo diferencial.
- **Omnidirecional:** com rodas Mecanum, permite deslocamentos longitudinais e laterais independentes da orientação do robô.
- **Ackermann:** direção do tipo automóvel, adequada à validação de algoritmos para veículos não-holonômicos.

Neste trabalho o robô é utilizado na configuração **diferencial**. Nesse modo, as velocidades linear v e angular ω do robô relacionam-se com as velocidades angulares das rodas esquerda (ω_L) e direita (ω_R) por [8]:

$$v = \frac{(\omega_L + \omega_R) r_w}{2}, \quad \omega = \frac{(\omega_R - \omega_L) r_w}{b}, \quad (1)$$

onde r_w é o raio da roda e b a distância entre as rodas de lados opostos. O controlador de baixo nível embarcado no LIMO converte automaticamente os comandos de alto nível $u = [v \ \omega]^T$ — publicados no tópico `/cmd_vel` — nos sinais PWM enviados aos motores, de modo que o desenvolvedor trabalha diretamente no nível das velocidades linear e angular.

2.1.2 Computador de Bordo e Software

O processamento embarcado é realizado por uma placa **NVIDIA Jetson Nano**, que integra uma CPU quad-core ARM Cortex-A57 de 64 bits a 1,43 GHz, 4 GB de memória RAM LPDDR4 e uma GPU Maxwell de 128 núcleos com suporte a aceleração de aprendizado profundo [8]. Essa GPU é o que viabiliza a execução de redes neurais de detecção, como o YOLO, diretamente a bordo do robô. A alimentação é feita por uma bateria de 12 V e 5200 mAh, com autonomia de aproximadamente 40 minutos em operação ativa. O robô conta ainda com uma unidade de medição inercial (IMU) de 6 graus de liberdade e um chassi metálico de cerca de 4,8 kg que suporta até 4 kg de carga útil.

Em software, a Jetson Nano executa **Ubuntu 18.04** com o SDK NVIDIA JetPack. O driver do robô é fornecido pela AgileX como um pacote ROS (`limo_base`), responsável pela comunicação com o chassi, pela publicação da odometria e pela recepção dos comandos de velocidade. O pacote `limo_bringup` reúne os *launchers* que inicializam o driver, o LiDAR e a árvore de transformadas (*tf*) entre a base, a câmera, a IMU e o laser [8].

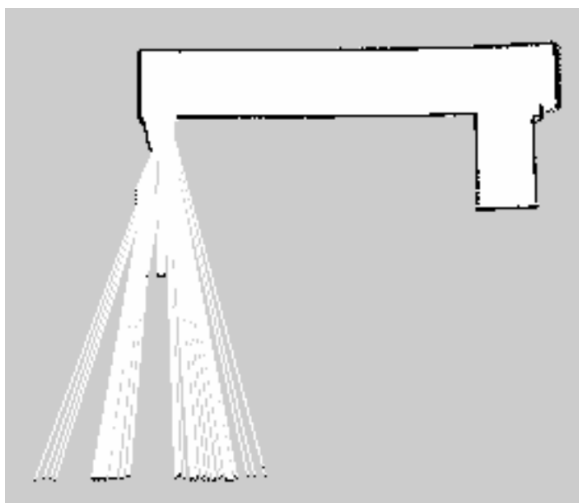
2.1.3 LiDAR — EAI T-Mini Pro

O sensor LiDAR utilizado é o **EAI T-Mini Pro (YDLidar)**, que realiza varreduras angulares do ambiente emitindo feixes de laser e medindo o tempo de retorno de cada pulso. O resultado é um conjunto de pares (θ, d) , onde θ é o ângulo e d é a distância até o obstáculo mais próximo naquela direção, formando um mapa de pontos 2D do ambiente.

As principais características relevantes para este projeto são:

- Ângulo de varredura: -90 a $+90$ (frontal)
- Frequência de atualização: ≈ 9 Hz
- Distância mínima: 0,15 m
- Distância máxima: 12 m

As Figuras 2a e 2b mostram mapas de pontos gerados pelo LiDAR em diferentes ambientes.



(a) Mapa de pontos — corredor



(b) Mapa de pontos — laboratório

Figura 2: Varreduras do LiDAR EAI T-Mini Pro em diferentes ambientes.

2.1.4 Câmera — Orbbec Astra Dabai U3

A câmera utilizada é a **Orbbec Astra Dabai U3**, uma câmera RGB-D (colorida com profundidade) com campo de visão horizontal de 60 e resolução de 640×480 pixels a 30 fps. Neste projeto é utilizado apenas o canal RGB, publicado no tópico `/camera/rgb/image_raw`. A figura 3 mostra o tópico sendo monitorado via Foxglove Studio durante os testes.



Figura 3: Visualização do tópico `/camera/rgb/image_raw` no Foxglove Studio.

2.2 Robot Operating System (ROS)

O Robot Operating System (ROS) é um arcabouço de software (*framework*) de código aberto voltado ao desenvolvimento de aplicações robóticas [5]. Apesar do nome, não se trata de um sistema operacional convencional, mas de um *meta*-sistema operacional que roda sobre o Linux e provê os serviços que se esperaria de um sistema operacional no contexto da robótica: abstração de *hardware*, controle de dispositivos em baixo nível, troca de mensagens entre processos e gerenciamento de pacotes, além de um conjunto de ferramentas e bibliotecas para obtenção, compilação e execução de código distribuído entre múltiplos computadores [4].

A principal motivação do ROS é facilitar a integração e o reúso de software em robótica. Em vez de um único programa monolítico, o sistema é decomposto em vários blocos independentes — os **nós** (*nodes*) —, cada um responsável por uma tarefa pequena e bem definida, como a aquisição de dados de um sensor, uma malha de controle ou o acionamento de um atuador. Essa modularidade permite desenvolver, testar e substituir cada componente isoladamente, característica diretamente explorada neste trabalho, no qual cada etapa do *pipeline* é implementada como um nó separado.

Neste projeto é utilizado o **ROS 1 Melodic**, rodando sobre Ubuntu 18.04 no *hardware* embarcado (NVIDIA Jetson Nano) do LIMO.

2.2.1 Arquitetura *Publish-Subscribe*

A comunicação entre os nós segue majoritariamente o padrão *publish-subscribe*. Um nó **publica** (*publisher*) mensagens em um **tópico** nomeado, e qualquer outro nó interessado pode **assinar** (*subscriber*) esse tópico para recebê-las, sem que haja acoplamento direto entre

eles: o publicador não sabe quais nós são os assinantes, e vice-versa. Esse desacoplamento é o que confere flexibilidade ao sistema — foi ele, por exemplo, que permitiu reaproveitar o nó `lidar_reader` no modo de tarefas apenas remapeando o tópico de entrada, sem alterar seu código.

O registro e a descoberta dos nós e tópicos são coordenados por um processo mestre, o `roscore` (ROS *Master*). É ele quem permite que um publicador e um assinante se localizem; após esse contato inicial, a troca de mensagens ocorre diretamente entre os nós. A Figura 4 ilustra o conceito.

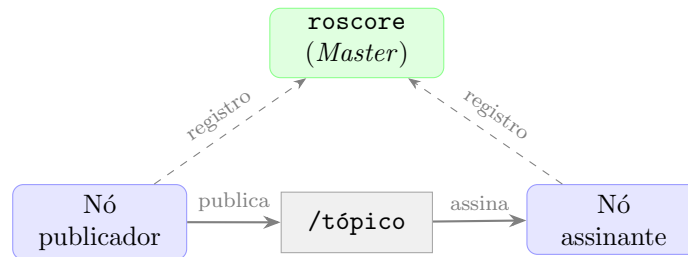


Figura 4: Padrão *publish-subscribe* do ROS: o `roscore` coordena o registro dos nós (linhas tracejadas) e as mensagens trafegam pelo tópico do publicador ao assinante (linhas cheias).

2.2.2 Conceitos Fundamentais

Os principais conceitos do ROS utilizados neste trabalho são:

- **Nós (*Nodes*)**: processos independentes, cada um responsável por uma tarefa específica. Neste projeto, cada etapa do *pipeline* (detecção, leitura do LiDAR, fusão, controle) é um nó separado.
- **Tópicos (*Topics*)**: canais de comunicação assíncronos pelos quais os nós trocam mensagens, no padrão *publish-subscribe*.
- **Mensagens (*Messages*)**: estruturas de dados tipadas que trafegam pelos tópicos. Exemplos usados neste projeto: `sensor_msgs/LaserScan` para dados do LiDAR, `sensor_msgs/Image` para *frames* da câmera e `geometry_msgs/Twist` para comandos de velocidade.
- **Master (*roscore*)**: processo central que registra nós e tópicos e viabiliza que publicadores e assinantes se encontrem.
- **Pacotes (*Packages*)**: unidade de organização do código, reunindo nós, bibliotecas, arquivos de configuração e *launchers*. Todo o código deste trabalho está organizado no pacote `follower_limo`.

2.2.3 Ferramentas Utilizadas

O ROS oferece um conjunto de utilitários de linha de comando que apoiam o desenvolvimento e a depuração. Os mais utilizados neste trabalho foram: o `roscore`, que inicia o *Master*; o `roslaunch` e o `roslaunch`, que executam nós individualmente ou em conjunto a partir de arquivos `.launch`; e o `rostopic`, que permite inspecionar, publicar e medir a frequência das mensagens em um tópico. A visualização dos dados em tempo real — imagens da câmera, detecções e varreduras do LiDAR — foi feita com o Foxglove Studio, conectado ao ROS por meio do `foxglove_bridge`.

2.3 Detecção por Cor (HSV) e Fusão Sensorial

2.3.1 Espaço de Cor HSV

A primeira abordagem de percepção deste trabalho baseia-se na segmentação por cor. Embora as câmeras forneçam a imagem no espaço RGB (ou BGR, no OpenCV), esse espaço mistura, em seus três canais, tanto a informação cromática quanto a luminosidade, o que o torna sensível a variações de iluminação. Por esse motivo, adota-se o espaço **HSV** (*Hue*, *Saturation*, *Value*), que separa a cor em três componentes com significado perceptual [15]:

- **Matiz** (*Hue*): o tom da cor, representado como um ângulo no círculo cromático (0 a 360);
- **Saturação** (*Saturation*): a pureza ou intensidade da cor;
- **Valor** (*Value*): o brilho.

A vantagem prática é que a identidade cromática de um objeto concentra-se principalmente no canal de matiz, relativamente independente do brilho. Assim, a segmentação torna-se mais robusta a mudanças de iluminação do que seria no espaço RGB. A detecção consiste em construir uma máscara binária $M(x, y)$ que isola os *pixels* cujos valores caem dentro de faixas pré-definidas:

$$M(x, y) = \begin{cases} 1, & \text{se } H_{\min} \leq H \leq H_{\max} \wedge S_{\min} \leq S \leq S_{\max} \wedge V_{\min} \leq V \leq V_{\max}, \\ 0, & \text{caso contrário.} \end{cases} \quad (2)$$

Cores que cruzam a origem do círculo de matiz — como o vermelho, situado simultaneamente próximo a 0 e a 360 — exigem duas faixas de matiz combinadas. Após a limiarização, operações morfológicas removem ruído e o maior contorno válido é tomado como o alvo. Os detalhes de implementação deste processo são apresentados na Seção 3.1.

2.3.2 Fusão Sensorial

A detecção por cor localiza o alvo *angularmente* na imagem, mas não informa a que *distância* ele está. Câmeras RGB, isoladamente, carecem de precisão métrica de profundidade; o LiDAR, por outro lado, mede distâncias com exatidão, mas não distingue a natureza ou a cor dos objetos. A **fusão sensorial** combina essas modalidades complementares para obter uma estimativa mais completa e confiável do que qualquer sensor forneceria sozinho [2].

A literatura costuma classificar a fusão em três níveis: no nível de dados brutos, no nível de características (*feature-level*) e no nível de decisão. Neste trabalho emprega-se a fusão em **nível de características**: extrai-se o ângulo do alvo a partir da câmera e a distância a partir do LiDAR naquela mesma direção, unindo-os em uma estimativa geométrica única da posição relativa do alvo. Para atenuar ruídos e saltos nas leituras, aplica-se ainda um filtro de suavização exponencial, detalhado na Seção 3.1. Essa estratégia de redundância eleva a robustez do sistema: falhas momentâneas de um sensor — como uma oclusão parcial ou uma variação brusca de iluminação — são mitigadas pela informação do outro [3].

2.4 YOLO

A detecção de objetos é a tarefa de visão computacional que busca responder a duas perguntas sobre uma imagem: *quais* objetos existem e *onde* eles estão localizados [13]. O YOLO (*You Only Look Once*) é um algoritmo de detecção de objetos baseado em aprendizado profundo, proposto originalmente por [9]. Sua principal inovação foi tratar a detecção como um

único problema de regressão, resolvido por uma única rede neural convolucional que, em uma só passagem (*single-stage*), prevê simultaneamente as caixas delimitadoras (*bounding boxes*) e as classes dos objetos presentes na imagem. Essa abordagem contrasta com os detectores de dois estágios da família R-CNN, que primeiro geram regiões candidatas e só depois as classificam, sendo significativamente mais lentos. Por unificar as duas etapas, o YOLO alcança desempenho em tempo real, característica essencial para aplicações robóticas embarcadas.

2.4.1 Divisão em Grade e Predições

O YOLO divide a imagem de entrada em uma grade de $S \times S$ células. Cada célula é responsável por detectar objetos cujo centro recai sobre ela, prevendo B caixas delimitadoras e uma pontuação de confiança para cada uma. Cada caixa é descrita por cinco valores: as coordenadas do centro e as dimensões (x, y, w, h) e a confiança associada.

A pontuação de confiança reflete tanto a probabilidade de a caixa conter de fato um objeto quanto a precisão geométrica da predição, sendo definida como:

$$\text{Confiança} = \text{Pr}(\text{Objeto}) \cdot \text{IoU}_{\text{predição}}^{\text{verdade}}, \quad (3)$$

onde $\text{Pr}(\text{Objeto}) \in \{0, 1\}$ indica a existência de um objeto na célula e a IoU (*Intersection over Union*) mede a sobreposição entre a caixa prevista B_p e a caixa de referência B_v :

$$\text{IoU} = \frac{\text{área}(B_p \cap B_v)}{\text{área}(B_p \cup B_v)}. \quad (4)$$

Além das caixas, cada célula prevê C probabilidades condicionais de classe, $\text{Pr}(\text{Classe}_i | \text{Objeto})$. No momento da inferência, a probabilidade condicional é multiplicada pela confiança da caixa, produzindo uma pontuação específica por classe que combina a probabilidade da classe com a qualidade do ajuste da caixa:

$$\text{Pr}(\text{Classe}_i | \text{Objeto}) \cdot \text{Pr}(\text{Objeto}) \cdot \text{IoU}_{\text{predição}}^{\text{verdade}} = \text{Pr}(\text{Classe}_i) \cdot \text{IoU}_{\text{predição}}^{\text{verdade}}. \quad (5)$$

O resultado é codificado em um tensor de saída de dimensão:

$$S \times S \times (B \cdot 5 + C). \quad (6)$$

A título de exemplo, na formulação original com $S = 7$, $B = 2$ e $C = 20$ classes, obtém-se um tensor $7 \times 7 \times 30$.

2.4.2 Arquitetura

A arquitetura do YOLO pode ser dividida em três componentes principais [11]:

- **Backbone:** rede convolucional que extrai características (*features*) da imagem em múltiplas escalas, empilhando camadas convolucionais.
- **Neck:** processa e combina os mapas de características de diferentes níveis, garantindo informação multi-escala.
- **Head:** a partir das características fundidas, prevê as caixas delimitadoras e as probabilidades de classe. A versão YOLOv3 introduziu o uso de *anchor boxes* em múltiplas escalas na *head*, melhorando a detecção de objetos de tamanhos variados [12].

O algoritmo possui duas variantes básicas: o modelo completo (*Regular*) e o modelo reduzido (*Tiny*). Enquanto o modelo completo do YOLO original emprega 24 camadas convolucionais seguidas de 2 camadas totalmente conectadas, a versão *Tiny* utiliza um número muito menor de camadas, sacrificando parte da precisão em troca de uma inferência consideravelmente mais rápida e de menor custo computacional. Essa característica torna a versão *Tiny* adequada a plataformas embarcadas com recursos limitados, como a Jetson Nano do LIMO.

2.4.3 Supressão Não-Máxima (NMS)

Como várias células podem prever caixas para um mesmo objeto, gera-se um conjunto redundante de detecções. Para produzir uma saída limpa, aplica-se a supressão não-máxima (*Non-Maximum Suppression*, NMS): as caixas são ordenadas por confiança e, iterativamente, mantém-se a de maior pontuação enquanto se descartam as demais que apresentem IoU acima de um limiar τ_{NMS} em relação a ela, eliminando duplicatas e detecções de baixa confiança.

2.4.4 Função de Custo

O treinamento minimiza uma função de custo composta por um termo de localização $\mathcal{L}_{\text{loc}}$, que penaliza erros na posição e nas dimensões das caixas, e um termo de classificação $\mathcal{L}_{\text{cls}}$, que penaliza erros nas probabilidades de classe [9]:

$$\mathcal{L} = \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{cls}}. \quad (7)$$

2.4.5 YOLO neste Projeto

Neste trabalho utiliza-se o **YOLOv3-tiny** no formato *darknet*, carregado através do módulo `cv2.dnn` da biblioteca OpenCV. Essa escolha decorre de restrições de compatibilidade do ambiente embarcado: o robô opera com Python 3.6 e OpenCV 4.1.1, versões que impossibilitam o uso de *frameworks* mais recentes. O modelo é pré-treinado sobre o conjunto de dados MS-COCO, que contém 80 classes de objetos, permitindo detectar categorias como *sports ball*, *person*, *laptop* e *bottle* sem necessidade de novo treinamento.

2.5 Campos Potenciais

O método dos campos potenciais artificiais, proposto por [14], é uma técnica clássica de navegação reativa amplamente empregada em robótica móvel — inclusive nas próprias aplicações de navegação do LIMO em ambientes congestionados [8]. A ideia central é modelar o robô como uma partícula sujeita a um campo de potencial artificial, no qual o objetivo exerce uma **força atrativa** e os obstáculos exercem **forças repulsivas**. O movimento resulta da soma vetorial dessas forças, conduzindo o robô até o alvo enquanto o afasta de colisões.

O potencial total $U(q)$ que atua sobre o robô na posição q é dado pela soma das componentes atrativa e repulsiva:

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q), \quad (8)$$

e a força resultante é o negativo do seu gradiente:

$$\vec{F}(q) = -\nabla U(q) = \vec{F}_{\text{att}}(q) + \vec{F}_{\text{rep}}(q). \quad (9)$$

2.5.1 Força Atrativa

O potencial atrativo cresce com a distância ao alvo, sendo comumente definido de forma parabólica:

$$U_{\text{att}}(q) = \frac{1}{2} k_{\text{att}} \rho^2(q, q_{\text{alvo}}), \quad (10)$$

onde k_{att} é o ganho atrativo e $\rho(q, q_{\text{alvo}})$ é a distância entre o robô e o alvo. A força atrativa correspondente aponta na direção do alvo, com magnitude proporcional a essa distância:

$$\vec{F}_{\text{att}}(q) = -\nabla U_{\text{att}}(q) = -k_{\text{att}} (q - q_{\text{alvo}}). \quad (11)$$

2.5.2 Força Repulsiva

A força repulsiva só atua quando o robô se aproxima de um obstáculo a uma distância inferior a um raio de influência ρ_0 ; além desse limite, o obstáculo é ignorado. O potencial repulsivo é dado por:

$$U_{\text{rep}}(q) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right)^2, & \text{se } \rho \leq \rho_0, \\ 0, & \text{se } \rho > \rho_0, \end{cases} \quad (12)$$

onde k_{rep} é o ganho repulsivo e ρ é a distância ao obstáculo mais próximo. A força cresce rapidamente à medida que o robô se aproxima do obstáculo, tendendo ao infinito quando $\rho \rightarrow 0$, o que garante o comportamento de desvio.

2.5.3 Aplicação neste Projeto

No modo de tarefas deste trabalho, a técnica é aplicada de forma simplificada para o controle da velocidade linear durante a aproximação de um objeto detectado pelo YOLO. A componente atrativa é proporcional ao erro entre a distância medida d e a distância desejada d_{alvo} , enquanto a componente repulsiva segue a formulação de [14], considerando a distância d ao obstáculo frontal medida pelo LiDAR:

$$f_{\text{att}} = k_p (d - d_{\text{alvo}}), \quad f_{\text{rep}} = \begin{cases} k_R \left(\frac{1}{d} - \frac{1}{d_0} \right), & \text{se } d < d_0, \\ 0, & \text{caso contrário,} \end{cases} \quad (13)$$

resultando na velocidade linear de comando:

$$v = f_{\text{att}} - f_{\text{rep}}. \quad (14)$$

Dessa forma, quando um obstáculo surge entre o robô e o alvo, o termo repulsivo reduz ou até inverte a velocidade de avanço, evitando a colisão. Os detalhes de implementação, incluindo os valores dos ganhos e do raio de influência, são apresentados na Seção 3.2.

3 Sistema Desenvolvido

O sistema foi desenvolvido em dois modos de operação independentes. O **Modo 1** realiza o seguimento de um alvo detectado por cor, combinando câmera e LiDAR via fusão sensorial. O **Modo 2** estende o sistema com detecção de objetos por categoria usando a rede YOLO, uma biblioteca de ações primitivas, freio de segurança e aproximação por campos potenciais. Ambos os modos publicam comandos no tópico `/cmd_vel`. O código-fonte foi desenvolvido em Python e está disponibilizado em um repositório Git, sob a referência [7].

3.1 Modo 1 — Seguidor por Cor com Fusão Sensorial

O Modo 1 é composto por quatro nós ROS que formam um pipeline sequencial, ilustrado na Figura 5.

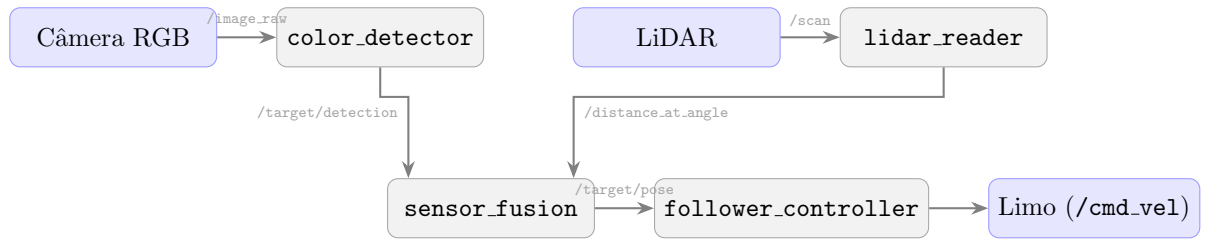


Figura 5: Pipeline do sistema de seguimento: sensores, nós de processamento e tópicos ROS.

3.1.1 Nó 1 — color_detector

Função: detectar o alvo colorido na imagem da câmera e calcular seu ângulo horizontal em relação ao centro do campo de visão.

Subscreve: `/camera/rgb/image_raw` (`sensor_msgs/Image`)

Publica:

- `/target/detection` — vetor `[found, cx_norm, cy_norm, area, angle_deg]`
- `/target/image_debug` — imagem anotada com *bounding box* para visualização

Método: segmentação por cor no espaço HSV com OpenCV. O frame é convertido de BGR para HSV, aplica-se uma máscara de cor para isolar o alvo, operações morfológicas removem ruído e o maior contorno válido é selecionado. O centro do contorno é convertido em ângulo usando o campo de visão da câmera:

$$\theta_{alvo} = \frac{c_x - W/2}{W/2} \cdot \frac{HFOV}{2} \quad (15)$$

onde c_x é a posição horizontal do centro do alvo em pixels e W a largura da imagem.

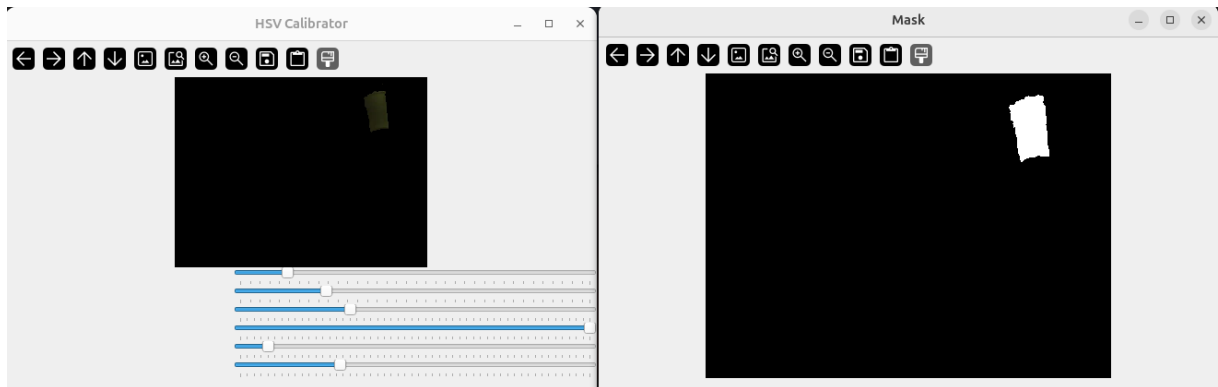
Tabela 1: Parâmetros de segmentação cromática e geometria da câmera.

Parâmetro (Unidade)	Descrição detalhada
Limiares HSV Primários	Faixas inferior e superior de Matiz, Saturação e Valor para isolar a cor alvo.
Limiares HSV Secundários	Segunda faixa opcional de cores (utilizada para cores que cruzam o limite do círculo HSV, como o vermelho).
Área Mínima do Contorno (px^2)	Filtro de área que desconsidera pequenos agrupamentos de pixels para eliminar ruídos visuais.
HFOV da Câmera (graus)	Campo de visão horizontal do sensor, utilizado para converter a posição do pixel em ângulo real.

A calibração dos valores HSV foi realizada com uma ferramenta auxiliar (`hsv_calibrator.py`) que permite ajustar interativamente os limiares sobre uma imagem real capturada pela câmera do robô. As Figuras 6a e 6b ilustram o processo de calibração.



(a) Imagem bruta capturada pela câmera do LIMO



(b) Interface do calibrador HSV com máscara isolando o alvo

Figura 6: Processo de calibração HSV para o alvo (copo verde neon).

3.1.2 Nó 2 — `lidar_reader`

Função: dado o ângulo calculado pela câmera, retornar a distância medida pelo LiDAR naquela direção.

Subscreve:

- `/scan` (`sensor_msgs/LaserScan`) — varredura contínua do LiDAR
- `/target/detection` — ângulo do alvo publicado pelo `color_detector`

Publica: `/lidar/distance_at_angle` — vetor `[angle_deg, distance_m, valid]`

Método: o ângulo em graus é convertido para o índice correspondente no array de ranges do `LaserScan`. Para robustez, a distância é calculada como a mediana de uma janela de 5 leituras vizinhas, descartando valores fora dos limites válidos e leituras NaN/Inf.

Tabela 2: Parâmetros de filtragem e leitura do sensor LiDAR.

Parâmetro (Unidade)	Descrição detalhada
Faixa Angular Válida (graus)	Define o campo de visão frontal considerado para busca (ex: $\pm 60^\circ$ em relação ao eixo central).
Distância Mínima Válida (m)	Limite inferior para ignorar leituras muito próximas ao próprio corpo do robô ou ruídos.
Distância Máxima Válida (m)	Limite superior para desconsiderar objetos muito distantes que não fazem parte do cenário de interesse.

3.1.3 Nó 3 — sensor_fusion

Função: combinar direção (câmera) e distância (LiDAR) em uma estimativa suavizada da posição relativa do alvo, entregando ao controlador os erros prontos para uso.

Subscreve:

- /target/detection
- /lidar/distance_at_angle

Publica: /target/pose — vetor [found, angle_deg, distance_m, error_angular_norm, error_distance_m]

Método: a fusão é geométrica e direta (*feature-level fusion*). Os valores brutos dos sensores são suavizados com um filtro exponencial:

$$\hat{x}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \hat{x}_{t-1} \quad (16)$$

O erro angular normalizado $e_\theta \in [-1, 1]$ alimenta o controle de rotação, e o erro de distância $e_d = d - d_{desejada}$ alimenta o controle de avanço.

Tabela 3: Parâmetros de configuração do nó de fusão sensorial.

Parâmetro (Unidade)	Descrição detalhada
Distância Alvo (m)	Valor de referência que o robô deve manter em relação ao objeto detectado.
Timeout do Alvo (s)	Tempo máximo de persistência do dado de fusão antes de invalidar a posição.
Fator de Suavização Angular (α)	Coefficiente do filtro passa-baixa para suavizar as variações bruscas no ângulo do alvo.
Fator de Suavização de Distância (α)	Coefficiente aplicado para filtrar ruídos e saltos nas leituras de distância do LiDAR.

3.1.4 Nó 4 — follower_controller

Função: converter os erros calculados pela fusão em comandos de velocidade para o robô.

Subscreve: /target/pose

Publica: /cmd_vel (geometry_msgs/Twist)

Método: controlador proporcional com três modos de operação:

1. **Seguimento normal** — alvo visível: rotação proporcional ao erro angular; avanço/recuo proporcional ao erro de distância (somente quando já alinhado).
2. **Recovery behavior** — alvo perdido: o robô gira lentamente no último sentido em que viu o alvo, tentando reencontrá-lo por até `recovery_timeout` segundos.
3. **Parada de segurança** — se o tópico `/target/pose` parar de ser publicado por qualquer razão (falha de nó, etc.), um timer independente zera `/cmd_vel` em até 0,5 s.

$$\omega = -K_{p,\omega} \cdot e_\theta \quad v = K_{p,v} \cdot e_d \cdot \mathbf{1}_{|e_\theta| < 0,3} \quad (17)$$

Tabela 4: Parâmetros de configuração e controle do sistema de seguimento.

Parâmetro (Unidade)	Descrição detalhada do comportamento
Ganho Angular (K_p)	Define a intensidade da correção de rotação para alinhar o robô ao centro do alvo.
Ganho Linear (K_p)	Define a velocidade de aproximação ou recuo baseada no erro de distância.
Velocidade Linear Máxima (m/s)	Limite superior de segurança para deslocamentos frontais e traseiros.
Velocidade Angular Máxima (rad/s)	Limite superior para a velocidade de rotação no próprio eixo.
Zona Morta Angular	Tolerância de erro de alinhamento para evitar oscilações constantes.
Zona Morta de Distância (m)	Faixa de tolerância em torno da distância alvo de 1 metro.
<i>Timeout</i> de Segurança (s)	Intervalo máximo sem detecção antes da interrupção total dos motores (<i>E-stop</i>).
Velocidade de Recuperação (rad/s)	Velocidade constante aplicada para busca ativa em 360° após a perda do alvo.
<i>Timeout</i> de Busca (s)	Tempo limite para o comportamento de rotação antes de retornar ao estado de repouso.

3.2 Modo 2 — Tarefas com Detecção YOLO

O Modo 2 substitui a detecção por cor por detecção de objetos por categoria, usando a rede YOLO, e introduz uma camada de execução de tarefas. Em vez de um único comportamento de seguimento contínuo, o robô passa a executar sequências de **ações primitivas** (gitar, avançar, procurar, aproximar-se, alertar), descritas em uma lista JSON. Essa organização prepara o sistema para, futuramente, receber comandos em linguagem natural traduzidos por um modelo de linguagem, mas já é plenamente funcional recebendo a lista de ações diretamente. A arquitetura é ilustrada na Figura 7.

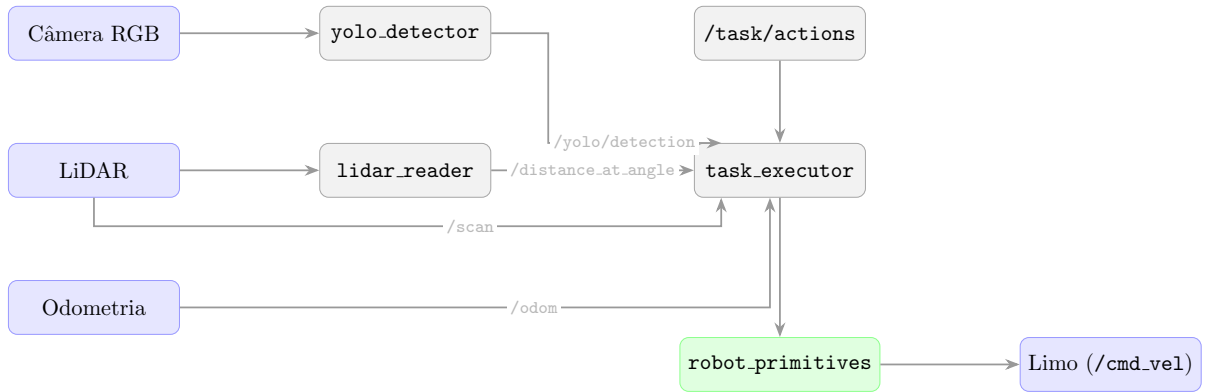


Figura 7: Arquitetura do modo de tarefas: percepção por YOLO, leitura do LiDAR e odometria alimentam o `task_executor`, que aciona as primitivas de movimento em `robot_primitives`.

3.2.1 Nó — `yolo_detector`

Função: detectar objetos de uma classe-alvo na imagem da câmera e calcular o ângulo horizontal do objeto em relação ao centro da imagem.

Subscreve: `/camera/rgb/image_raw` (`sensor_msgs/Image`)

Publica:

- `/yolo/detection` — vetor `[found, cx_norm, cy_norm, area, angle_deg]`, no mesmo formato de `/target/detection` do Modo 1
- `/yolo/class` — `String` com a classe detectada
- `/yolo/image_debug` — imagem anotada com a *bounding box*

Método: a rede YOLOv3-tiny é carregada em formato *darknet* com a função `readNetFromDarknet` (módulo `cv2.dnn`). Cada quadro é convertido em um *blob* 416×416 e submetido à rede; as saídas passam por filtragem por limiar de confiança e por supressão não-máxima (NMS) para eliminar detecções redundantes. A detecção de maior confiança da classe-alvo é convertida em ângulo pela mesma relação geométrica do Modo 1, aproveitando o campo de visão da câmera. Como a saída segue o formato do `color_detector`, o `lidar_reader` pode ser reutilizado sem alteração de código — apenas com o remapeamento de `/target/detection` para `/yolo/detection`.

Tabela 5: Parâmetros de configuração do detector YOLO.

Parâmetro (Unidade)	Descrição detalhada
Classe Alvo	Categoria do dataset COCO a ser monitorada (ex.: <i>sports ball, person, laptop</i>).
Limiar de Confiança	Confiança mínima para aceitar uma detecção; filtra predições fracas.
Limiar de NMS	Limiar de IoU usado na supressão não-máxima para descartar caixas sobrepostas.
HFOV da Câmera (graus)	Campo de visão horizontal, usado para converter a posição do objeto em ângulo.

3.2.2 Nó — `task_executor`

Função: manter o estado dos sensores atualizado e executar, em sequência, a lista de ações recebida.

Subscreve:

- `/yolo/detection` e `/yolo/class` — percepção visual
- `/lidar/distance_at_angle` — distância ao alvo detectado
- `/scan` (`sensor_msgs/LaserScan`) — LiDAR bruto, para o freio de segurança
- `/odom` (`nav_msgs/Odometry`) — odometria das rodas
- `/task/actions` (`String` com JSON) — lista de ações a executar

Publica: `/cmd_vel`, `/task/status` (estado atual: `IDLE`, `RUNNING`: `<ação>`, `ERROR`: ...) e `/task/alert`.

Método: os *callbacks* dos sensores mantêm um objeto de estado (`RobotState`) sempre atualizado. Ao receber uma lista de ações, o executor valida o JSON e itera sobre cada ação, chamando a primitiva correspondente da biblioteca `robot_primitives` e publicando o status a cada etapa. Uma ação especial `cancel` interrompe a sequência. A partir do `/scan` bruto, o nó calcula continuamente a distância mínima no setor frontal de $\pm 30^\circ$, valor central para o freio de segurança e para os campos potenciais.

3.2.3 Biblioteca `robot_primitives`

Diferentemente dos nós anteriores, `robot_primitives` não é um nó ROS, mas uma biblioteca importada pelo `task_executor`. Cada primitiva é uma função bloqueante que só retorna quando a ação termina ou o *timeout* estoura. A Tabela 6 resume as ações disponíveis.

Tabela 6: Ações primitivas disponíveis no modo de tarefas.

Ação	Descrição
<code>go_forward</code>	Avança uma distância dada; usa odometria se disponível e para se o freio de segurança disparar.
<code>rotate</code>	Gira um ângulo dado (positivo = esquerda); acumula ângulo via odometria para suportar rotações maiores que 360° .
<code>scan_area</code>	Rotação de 360° para varrer o ambiente.
<code>search_object</code>	Gira lentamente até encontrar a classe-alvo via YOLO ou atingir o <i>timeout</i> .
<code>approach_object</code>	Aproxima-se do objeto com controle proporcional no ângulo e campos potenciais na velocidade linear.
<code>stop</code>	Interrompe o movimento imediatamente.
<code>alert</code>	Registra uma mensagem e a publica em <code>/task/alert</code> .

3.2.4 Freio de Segurança

O freio de segurança está ativo nas primitivas `go_forward` e `approach_object`. A cada iteração do laço de controle, verifica-se a distância mínima frontal medida pelo LiDAR; se ela

cair abaixo de um limiar de segurança d_{seg} , o robô é imediatamente parado e a ação é abortada:

$$d_{\text{frontal}} < d_{\text{seg}} \Rightarrow v = 0. \quad (18)$$

Neste projeto adotou-se $d_{\text{seg}} = 0,20$ m. O mecanismo é independente da percepção visual, funcionando como uma camada de proteção mesmo que o YOLO falhe em detectar um obstáculo.

3.2.5 Campos Potenciais na Aproximação

A primitiva `approach_object` aplica a formulação de campos potenciais apresentada na Seção 2.5 para regular a velocidade linear. A força atrativa é proporcional ao erro entre a distância medida pelo LiDAR e a distância desejada, e a força repulsiva atua quando um obstáculo entra no raio de influência d_0 . A velocidade angular, por sua vez, segue um controle proporcional ao erro angular fornecido pelo YOLO ($e_\theta = c_{x,\text{norm}}$):

$$\begin{aligned} \omega &= -k_{p,\omega} e_\theta, \\ v &= \underbrace{k_p (d - d_{\text{alvo}})}_{f_{\text{att}}} - \underbrace{k_R \left(\frac{1}{d} - \frac{1}{d_0} \right)}_{f_{\text{rep}}, d < d_0}. \end{aligned} \quad (19)$$

Ambas as velocidades são saturadas por limites máximos de segurança. A Tabela 7 lista os valores adotados.

Tabela 7: Parâmetros da aproximação por campos potenciais.

Parâmetro	Valor	Descrição
k_p (ganho atrativo)	0,40	Intensidade da força de atração ao alvo.
k_R (ganho repulsivo)	0,15	Intensidade da força de repulsão de obstáculos.
d_0 (raio de influência)	0,50 m	Distância a partir da qual a repulsão passa a atuar.
d_{seg} (freio)	0,20 m	Distância mínima frontal antes da parada de emergência.
$k_{p,\omega}$ (ganho angular)	0,60	Ganho do controle proporcional de rotação.

3.2.6 Odometria

As primitivas `go_forward` e `rotate` utilizam o tópico `/odom` para medir deslocamento e rotação reais a partir dos *encoders* das rodas. O ângulo de guinada (*yaw*) é extraído diretamente do quatérnion de orientação, sem dependência da biblioteca *tf*:

$$\psi = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)). \quad (20)$$

Quando a odometria não está disponível (tópico silencioso), as primitivas retornam automaticamente à estimativa por tempo, sem necessidade de configuração manual, garantindo robustez do sistema.

4 Resultados Experimentais

4.1 Seguimento por Cor (Fusão Sensorial)

4.1.1 Detecção Visual do Alvo

A detecção por segmentação HSV foi validada com sucesso. A Figura 8 mostra o sistema em operação: o alvo (copo verde neon) é identificado corretamente, com *bounding box* verde delimitando o objeto, indicação do ângulo calculado e o texto de detecção ativa.

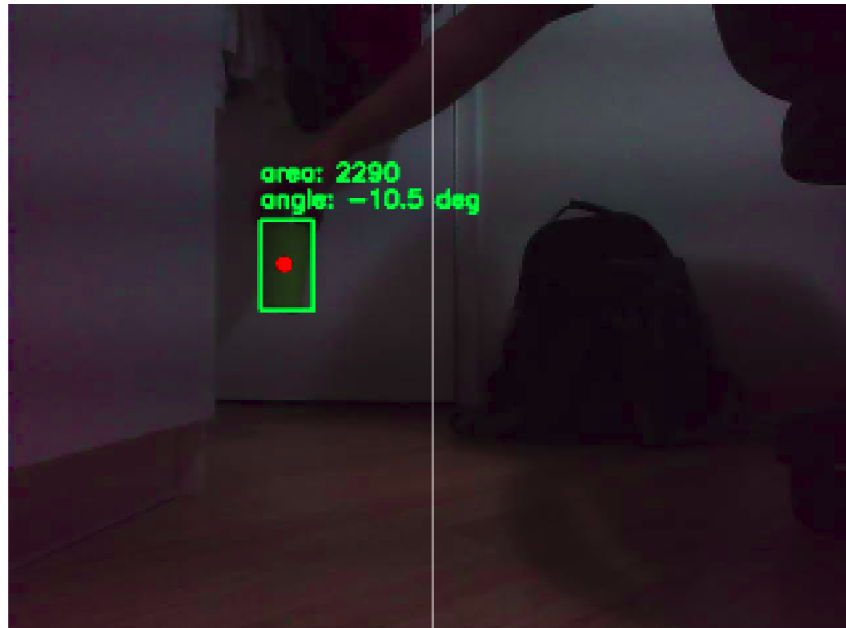


Figura 8: Sistema de detecção em operação: *bounding box* verde em volta do copo com ângulo calculado exibido em tempo real.

4.1.2 Fusão de Sensores

A fusão câmera + LiDAR foi validada com o nó `sensor_fusion` ativo. O trecho de log abaixo mostra a consistência dos dados dos dois sensores operando em conjunto:

Código 1: Log do sistema com fusão ativa

```
[INFO]: Alvo detectado | angle=12.8 deg | cx_norm=0.42 | area=2714  
[INFO]: LiDAR | angulo=12.8 deg | dist=3.04 m  
[INFO]: Fusao | ang=12.9 deg | dist=3.07 m | err_ang=0.43 | err_dist=2.07 m
```

A câmera e o LiDAR concordam no ângulo (12,8) e o LiDAR fornece a distância métrica (3,04 m). O erro de distância de 2,07 m indica que o alvo estava a ≈ 3 m enquanto a distância desejada é 1 m — comportamento correto e esperado.

4.1.3 Seguimento Autônomo

O sistema completo foi executado com sucesso. O robô demonstrou capacidade de seguir o alvo de forma contínua, mantendo a distância configurada de 1 metro e realizando rotação proporcional ao desvio angular. O comportamento de recuperação (*recovery behavior*) também foi observado: ao perder o alvo de vista, o robô gira lentamente no último sentido detectado até reencontrá-lo ou atingir o timeout de 4 segundos.

4.2 Detecção por YOLO

A detecção de objetos por categoria foi validada com o nó `yolo_detector` executando o modelo YOLOv3-tiny a bordo do LIMO. A Figura 9 mostra o sistema em operação, detectando corretamente um *laptop*: a *bounding box* verde delimita o objeto, o ponto vermelho marca seu centro e o rótulo exibe a classe, a confiança da detecção (46%) e o ângulo horizontal estimado em relação ao centro da imagem (3,3), próximo de zero por o objeto estar quase centralizado.

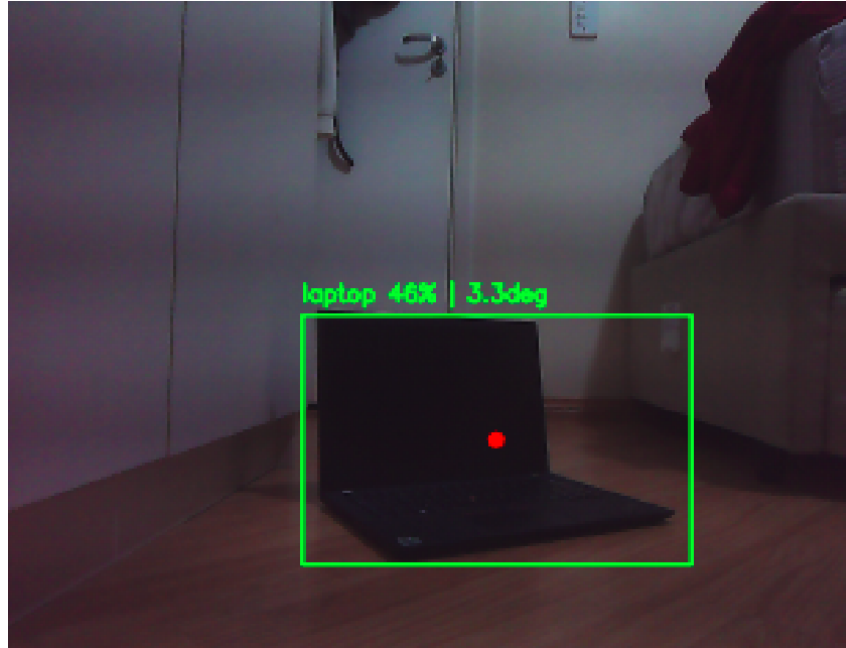


Figura 9: Detecção de um *laptop* pelo YOLOv3-tiny: *bounding box*, centro do objeto, classe, confiança e ângulo calculado exibidos em tempo real.

Observou-se que a qualidade das detecções é sensível às condições de iluminação do ambiente: em cenas com pouca luz, a confiança das predições cai e algumas classes tornam-se mais difíceis de detectar de forma estável. Esse comportamento é coerente com o uso da variante *tiny* da rede, que troca parte da precisão por velocidade de inferência para viabilizar a execução na Jetson Nano.

4.3 Aproximação com Campos Potenciais

[Seção a ser escrita.]

5 Conclusão

[Seção a ser escrita.]

Referências

- [1] PENDLETON, S. D. et al. Perception, planning, control, and coordination for autonomous vehicles. **Machines**, v. 5, n. 1, p. 6, 2017.
- [2] ZHANG, T. et al. Robust target tracking for mobile robots based on RGB-D and LiDAR sensor fusion. **Sensors**, v. 19, n. 5, p. 1258, 2019.
- [3] NGUYEN, V. D. et al. LiDAR-based Obstacle Avoidance and Target Following for Mobile Robots in Dynamic Environments. In: **ISKE**, 2020.
- [4] QUIGLEY, M.; GERKEY, B.; SMART, W. D. **Programming Robots with ROS**. O'Reilly Media, 2015.
- [5] QUIGLEY, M. et al. ROS: an open-source Robot Operating System. In: **ICRA Workshop on Open Source Software**, 2009.
- [6] AGILEX ROBOTICS. **LIMO User Manual**. 2024. Disponível em: <https://global.agilex.ai/>.
- [7] BONETTI, B. M. **follower_limo**: Implementação de fusão sensorial para seguimento de alvos em robôs LIMO (ROS1). 2024. Disponível em: https://github.com/bonettibruno/follower_limo.git.
- [8] PIMENTA, M. B. et al. Usando a Plataforma LIMO para Aprender Robótica. **Anais do Congresso Brasileiro de Automática (CBA)**, 2024.
- [9] REDMON, J.; DIVVALA, S.; GIRSHICK, R.; FARHADI, A. You Only Look Once: Unified, Real-Time Object Detection. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**, 2016.
- [10] REDMON, J.; FARHADI, A. **YOLOv3: An Incremental Improvement**. arXiv preprint arXiv:1804.02767, 2018.
- [11] ALI, M. L.; ZHANG, Z. The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection. **Computers**, v. 13, n. 12, 2024. DOI: 10.3390/computers13120336.
- [12] JIANG, P.; ERGU, D.; LIU, F.; CAI, Y.; MA, B. A Review of YOLO Algorithm Developments. **Procedia Computer Science**, v. 199, p. 1066–1073, 2022.
- [13] ZOU, Z.; CHEN, K.; SHI, Z.; GUO, Y.; YE, J. Object Detection in 20 Years: A Survey. **Proceedings of the IEEE**, v. 111, n. 3, p. 257–276, 2023.
- [14] KHATIB, O. Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. **The International Journal of Robotics Research**, v. 5, n. 1, p. 90–98, 1986.
- [15] GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4. ed. New York: Pearson, 2018.